

End-Project Submission Report

Praveer S. Chauhan (2023397)

Rishi Banerji (2023437)

Ujjwal Kumar (2023566)

1. Low Rank Adaptation

The original ToXCL framework finetunes the entire Flan-T5 model end to end during the training, Flan-T5 base model is approximately 248M parameters and fully updating them during fine tuning is expensive in 3 ways

1. GPU memory requirements scale with the number of trainable parameters since gradients and optimizer states must be stored for each parameter for example with AdamW, this means storing two additional momentum vectors per parameter, effectively tripling the memory of the model during training
2. Training time increases proportionally with the number of parameters being updated
3. Fine-tuning all parameters on a relatively small domain-specific dataset like IHC (15,635 training samples) risks overfitting

The teacher classifier in the original ToXCL paper, partially discusses this through knowledge distillation, but then that included its own overhead a fully trained RoBERTa large model of 354M parameters hence during training the pipeline would require Flan-T5 and RoBERTa and their respective optimizers into GPU memory

Therefore we used LoRA for finetuning without requiring an external teacher model

What LoRA does

Lora is based on the hypothesis that the weight updates for a fine tuning task need to be low rank that even if the original weight matrix is very large the fine tune change required can be approximated by a much smaller matrix

Formally, for a pre-trained weight matrix:

$$W \in \mathbb{R}^{(d \times k)}$$

Instead of directly computing and storing ΔW (which would be the same size as W), LoRA decomposes it as:

$$\Delta W = A \times B$$

The forward pass then becomes:

$$y = Wx + (\alpha/r) \times ABx$$

Where α is a scaling hyperparameter that controls how strongly the LoRA update influences the output relative to the frozen base weights. The division by r normalizes the update magnitude across different rank choices.

During training, W is completely frozen. Only A and B 's gradients are updated. This means the number of trainable parameters added per layer is $d \times r + r \times k$, compared to $d \times k$ for full fine-tuning. With $r = 8$ and typical Flan-T5 attention dimensions of $d = k = 512$, this reduces each layer's trainable parameters from 262,144 to just 8,192

Where we applied LoRA

We applied LoRA to the query and value projection matrix inside the self attention layers of the Flan-T5 encoder.

Query determines what each token searches for in the sequence. For toxic speech detection the model needs to learn to search for implicit signals, coded language, and stereotypical references that indicate toxicity

Value determines what information gets aggregated and passed forward once attention weights are computed. Even if the model attends to the right tokens, if W_v hasn't been adapted, the information extracted from those tokens may not be useful for classification

We only applied this to Flan-T5 encoder as the encoder is responsible for reading and understanding the input post

Flan -T5 has 12 encoder layers and each one containing a self attention block and LoRA was applied to all 12 layers

Total LoRA parameters = 12 layers $\times$ 2 matrices (q, v) $\times$ 2 LoRA matrices (A, B) = $12 \times 2 \times (512 \times 8 + 8 \times 512) = 12 \times 2 \times 8,192 = 196,608$ parameters which is much less than the 248M parameters

Before (standard Flan-T5 encoder layer):

Input X

- $W_q X = Q$ (262,144 params, all updated during training)
- $W_k X = K$ (262,144 params, all updated during training)
- $W_v X = V$ (262,144 params, all updated during training)
- Attention(Q, K, V)
- Output

After (LoRA-injected encoder layer):

Input X

- $(W_q + A_q B_q) X = Q$ (W_q frozen, only A_q, B_q trained 8,192 params)
- $W_k X = K$ (W_k frozen, untouched)

→ $(W_v + A_v B_v) X = V$ (W_v frozen, only A_v, B_v trained 8,192 params)
→ Attention(Q, K, V)
→ Output

Component	Parameters	Trainable
Flan-T5-base (full fine-tune)	248M	248M
Teacher Classifier (RoBERTa-Large)	354M	354M
Flan-T5-base with LoRA	248M + ~0.9M	0.9M

Code change

The integration required modifying only the model initialization in `toxcl.py`. The LoRA configuration was applied to the base Flan-T5 model before it was passed into the ToXCL class, using HuggingFace's PEFT library

The rest of the ToXCL architecture the classification head, the explanation decoder, the Conditional Decoding Constraint, and the loss computation remained completely unchanged

The loss function also simplifies in our no-teacher variant. The original total loss is:

$$\mathbf{L} = \lambda(\alpha \mathbf{L}_{\text{cls}} + \beta \mathbf{L}_{\text{clm}}) + \gamma \mathbf{L}_{\text{tf}}$$

In ToXCL-LoRA without teacher forcing, this reduces to:

$$\mathbf{L} = \alpha \mathbf{L}_{\text{cls}} + \beta \mathbf{L}_{\text{clm}}$$

Where $\mathbf{L}_{\text{cls}}$ is binary cross-entropy classification loss and $\mathbf{L}_{\text{clm}}$ is causal language modelling loss for explanation generation

Results:

Metric	Accuracy (Acc)	METEOR	BERTSCORE
LoRA	67.9448	67.9448	67.9448
Paper	76.5082	68.2694	68.7686

ToXCL-LoRA was trained for 3 epochs on the IHC dataset without teacher forcing. The model achieved a final accuracy of 67.94% on the validation set.

On Train Set:

Post	Ground Truth Label	Ground Truth Explanation	Ground Truth Target Groups	Predicted Label	Predicted Explanation	Predicted Target Groups
"as gregory hood wrote in his latest article on the british government is at least as oppressive as the soviet union or any of the people's republics ."	normal	none	Other	normal	none	Other, Refugee
"race baiting isn't a thing . and you're standing with a trump a known white supremacist . you're complicit in the oppression of black ppl"	hate	Black people are oppressed.	African, Caucasian	hate	Black people are oppressed	African, Caucasian
"fair has ties to white supremacists group . just reading their literature you can see how racist they are."	hate	White people are racist.	Caucasian	hate	White people are racist.	Caucasian, Other

The model correctly identifies hate speech in rows 2 and 3, generating target groups and explanations that align perfectly with the ground truth. It successfully classifies row 1 as normal, showing an understanding of context over simple keyword matching. For row 3, while the model identifies "Caucasian" correctly, it also predicts "Other" as a supplementary target. This suggests a tendency toward broader target-group labeling even when the ground truth is specific to a single group. For row 1, while the model correctly predicts the "normal" label, it identifies "Refugee" as a supplementary target group alongside "Other," indicating a tendency to associate political comparisons with broader societal themes even when no specific hate target is intended.

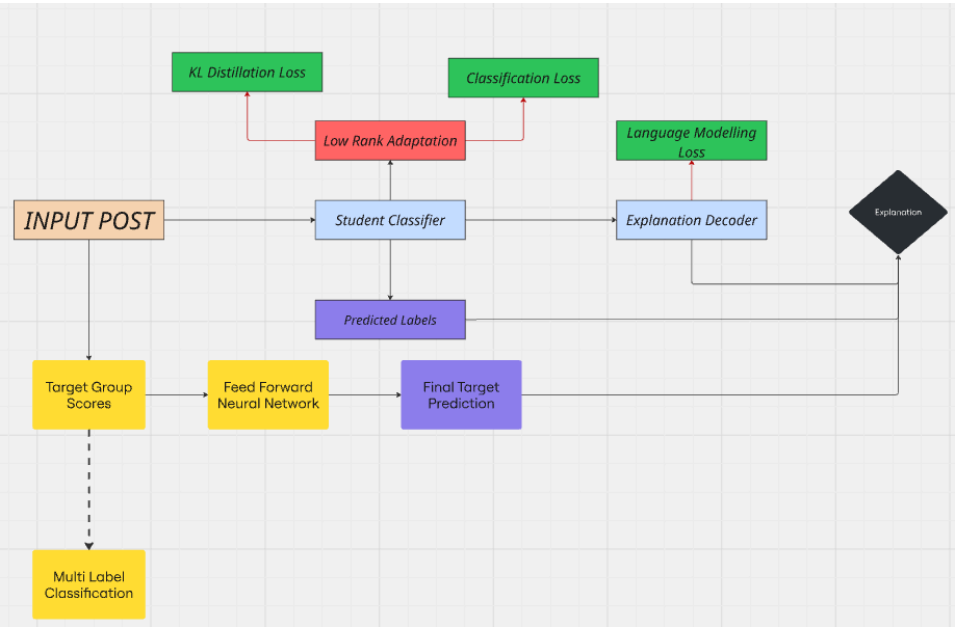
On Validation Set:

Post	Ground Truth Label	Ground Truth Explanation	Ground Truth Target Groups	Predicted Label	Predicted Explanation	Predicted Target Groups
------	--------------------	--------------------------	----------------------------	-----------------	-----------------------	-------------------------

"none of these people care about their followers . if baked al"	normal	none	normal	normal	none	None, Other Themes
"part of your white heritage is being ruled by jews that not part of asian heritage ."	hate	Jewish people want to control white people	Asian, Caucasian, Jewish	hate	jews hate white people	Asian, Caucasian, Jewish
"but yes there are whites who go along with the jewish agenda . they're commonly referred to as "" shabbos goys . "" "" "" ""	hate	Jewish people don't like whites.	Caucasian, Jewish	hate	Jewish are hateful of the white race	Caucasian, Jewish

The model correctly identifies hate speech in rows 2 and 3, generating Predicted Explanations and Target Groups that closely mirror the ground truth semantic meaning. It successfully classifies row 1 as normal, correctly suppressing the explanation to "none" despite the presence of potentially sensitive keywords. For row 1, the model correctly identifies the "normal" label but predicts "Other Themes" as a supplementary target group, suggesting a tendency to flag neutral content with broad thematic labels even when no specific hate target is present.

The model achieved high parametric efficiency by training only 0.9M parameters, but the drop in accuracy relative to the paper stems from the omission of the Teacher Classifier, which previously provided critical distillation guidance.



2. Soft Target Group Conditioning

The original ToXCL model uses target-group information by concatenating it with the post as text, so the input to the model is of the form: [Target: {target groups} Post: {post}]

This is useful because the paper shows that target-group information improves both toxicity detection and explanation generation. These are the components of the architecture in the paper: a target group generator, an encoder-decoder model for detection and explanation, a teacher classifier for distillation, and conditional decoding so that normal posts (posts which are not predicted as toxic) produce [None] instead of unnecessary explanations.

So in the paper, the generated target groups are only used in the prompt text. In addition to this in our approach, we use a soft target group conditioning method:

1. extract the target groups and make a target-group vocabulary using these
2. Learn embeddings for the target groups
3. Pool the embeddings into target group representations
4. Use these representations for classification and generation.

The representations are not given to the teacher classifier, but to the student classifier and the decoder. This is done using different ways:

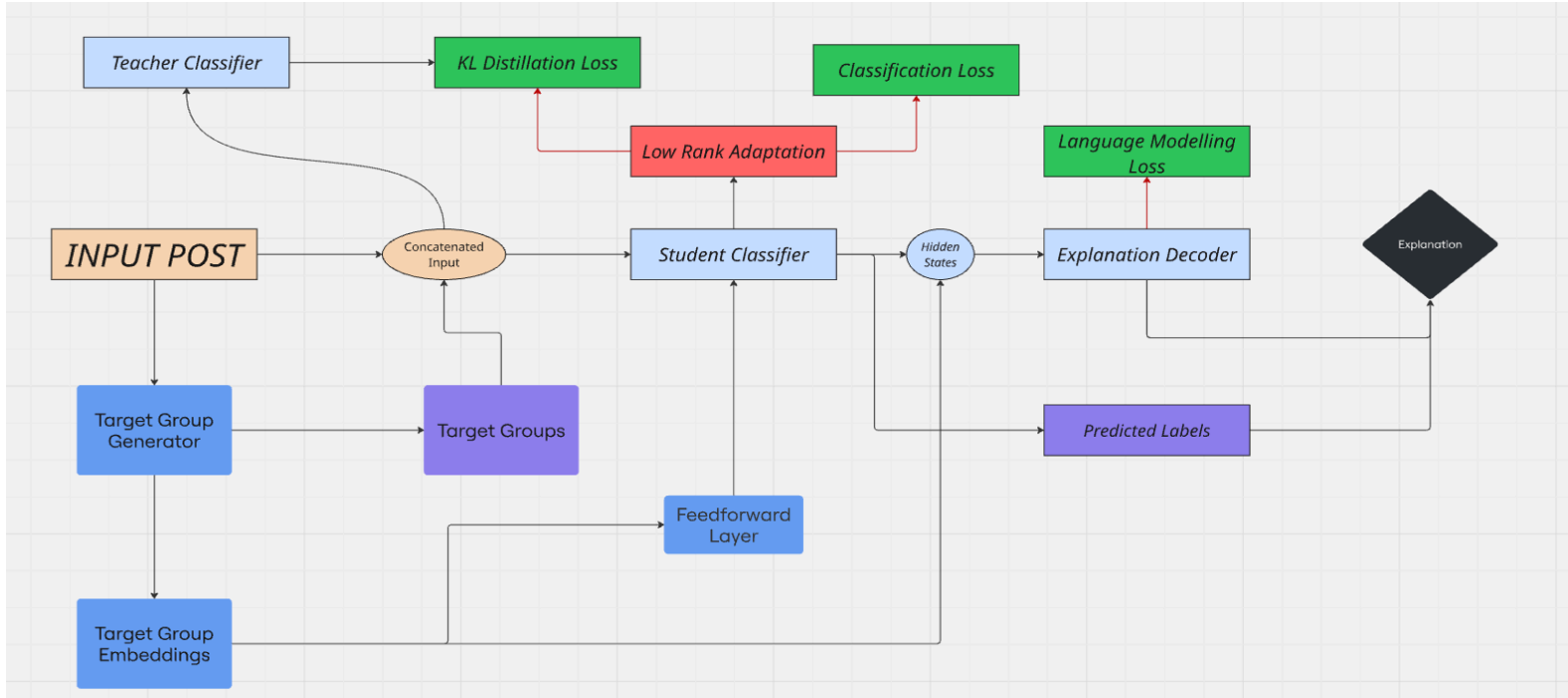
1. For the student classifier, the TG vector, concatenated with the other input meant for the student classifier, is passed through a linear layer. The output from this layer becomes the input for the student classifier.
2. Explanation decoder: Since this is an encoder-decoder based model, the decoder already gets hidden states from the encoder. The TG vector is appended to the encoder hidden states, and this combination is passed on to the decoder. As a result, the decoder can attend to the TG vector too, during generation.

1. Why soft conditioning

The paper established that target group information improved both classification and generation. Pure textual concatenation of TG info may not be the best way to use the info, so we thought it would be a good idea to improve this aspect of the model, and came up with two ways to do this:

1. Create embeddings for target groups.
2. Allow the decoder to attend to the target groups.

2. Architecture



3. Results

a) Results on val set

We trained the model on the IHC corpus. The results on the metrics (same as the ones used in the paper) are:

Metric	Accuracy (Acc)	METEOR	ROUGE-L
Soft-tg	69.61	67.89	67.94
Paper	76.51	68.27	68.35

Final losses were as follows:

Average valid LM loss: 4.0700

Average valid CLS loss: 0.5756

Average valid KL loss: 0.0176

b) Some examples from val set

Target & Post	Gold Label	Gold Explanation	Predicted Label (1 = hate, 0 = normal)	Predicted Explanation
"Target: none Post: people like you are directly responsible for this"	normal	none	0	none
"Target: Caucasian, Jewish Post: of course . unlike many white racists we aren't " mindless haters . " " the jewish " " finance shark " " isn't any worse than a white one "	hate	Jews are greedy money lovers.	1	jews are very greedy
Target: Refugee Post: well they could let them creep in like the illegals from so . of border !"	hate	Immigrants are inferior.	1	none

Overall (not just these 3 examples, the rest too), we observe that the model is better at classification than generation. In a few cases where the original text is hateful, the model correctly predicts it to be toxic, but doesn't generate anything.

In these examples, the model correctly identifies all the 3 posts. When it comes to generation, it correctly generates "none" for the first row (a non-hate example). It generates, "jews are very greedy" for the second row, which is a post targeted against Jews and Caucasians. This is quite close to the ground truth generation of "Jews are greedy money lovers.". For the third example, it generates "none" for a hateful post.

Also as observed from a greater set of val inference examples, among the misclassifications, the model has a tendency to produce false negatives more - predicting 0 when ground truth is 1.

By replacing text prompts with learned embeddings that the decoder can directly attend to, this method achieved higher accuracy than the LoRA version by creating a stronger mathematical link between specific target groups and toxic intent.

References

1. [Hoang et al. \(2024\)](#) - ToXCL: A Unified Framework for Toxic Speech Detection and Explanation.
2. [Caselli et al. \(2021\)](#) - HateBERT: Retraining BERT for Abusive Language Detection in English.
3. [ElSherief et al. \(2021\)](#) - Latent Hatred: A Benchmark for Understanding Implicit Hate Speech.
4. [Sap et al. \(2020\)](#) - Social Bias Frames: Reasoning about Social and Power Implications of Language
5. [HateXplain dataset](#) - Hate-speech-CNERG/hatexplain, Hugging Face Datasets.
6. [\[2106.09685\] LoRA: Low-Rank Adaptation of Large Language Models](#) - LoRA